

AgentMem v2 — Self-Improving Agent Memory & Decision System

Design Document | Version 2.0 | February 2026

1. Introduction

Modern AI agents fail under scale not because their reasoning is wrong, but because their memory is wrong. When memory is an unstructured vector blob, there is no causal chain between what was retrieved and what was decided. When there is no causal chain, there is no learning. When there is no learning, every decision is made cold — regardless of how many prior decisions were made correctly or incorrectly on the same subject.

This document describes **AgentMem v2**: a hybrid, self-improving agent memory system that separates memory *representation* (deterministic, structured) from memory *control* (learned, policy-driven). The system combines an entity-centric knowledge graph, soft semantic clustering, and a lightweight reinforcement-style controller — the **AgentMem Controller** — that decides at runtime how much to retrieve, what to store, and what to forget.

The design is production-extendable and weekend-feasible. All components are independently replaceable.

2. Tenets

In priority order. When they conflict, higher-order tenets win.

- 1. Auditability above convenience.** Every decision must be reproducible from its trace. A system that cannot explain itself cannot be trusted.
 - 2. Structure controls learning; learning controls memory.** The knowledge graph is never mutated by gradient descent. Only heuristics and human feedback write to it.
 - 3. Graceful degradation.** Every advanced subsystem (clustering, bandit) must have a working fallback so the system runs correctly from day one, without any trained components.
 - 4. Human corrections are first-class memory.** A human correction is not a log entry — it is a new fact that immediately updates the graph and closes the reward loop.
-

3. Problem Statement

3.1 What Fails Today

Failure Mode	Root Cause	Consequence
Stale facts influence decisions	No confidence decay or TTL	Agent acts on outdated knowledge
Retrieval noise poisons context	Fixed-depth traversal, no policy	Irrelevant facts crowd out relevant ones

Failure Mode	Root Cause	Consequence
No learning from outcomes	Trace logs exist but are never read	Same mistakes repeated indefinitely
Human corrections are ignored	Correction is logged, not ingested	Memory permanently wrong

3.2 What Success Looks Like

- A query on a topic with rich history retrieves *more* relevant context than a query on a novel topic — automatically, without manual tuning.
- After 50 decisions, the system's retrieval policy has measurably converged toward configurations that produced better outcomes.
- A human who types a correction sees that correction reflected in graph memory within seconds, not the next training cycle.
- Every decision can be replayed from its context trace, including which memory IDs were retrieved and in what order.

4. Architecture

4.1 High-Level Data Flow

```
flowchart TD
    A["User / News Monitor\n(Multimodal Inputs)"] --> B
    subgraph Canonicalization ["Canonicalization Layer"]
        B["Ingestor\n(ingest.py)"]
    end
    end

    B -->|"Entities + Triplets\n(confidence, source)"| C
    B -->|"Text embeddings"| E

    subgraph Memory ["Structured Memory Layer"]
        C["Graph Store\n(Neo4j)"]
        D["Cluster Store\n(HDBSCAN + FAISS)"]
        E["Vector Store\n(ChromaDB)"]
        F["Episodic Store\n(JSONL + outcomes)"]
    end
    end

    C <-->|"Cluster membership"| D

    subgraph Controller ["AgentMem Controller"]
        G["Policy\n(UCB Bandit)"]
        H["STORE gate\n(confidence threshold)"]
        I["FORGET\n(TTL decay)"]
    end
    end

    G -->|"depth, breadth params"| J
    H -->|"write decision"| C
```

```

I -->|"decay update"| C

subgraph Recall ["Candidate Recall Engine"]
    J["Retriever\n(retrieval.py)\n\n① Vector Search\n② Graph
Traversal\n③ Cluster Expansion"]
end

J --> K

subgraph Reasoning ["Reasoning Layer (AutoGen GroupChat)"]
    K["Planner Agent"]
    L["Memory Specialist"]
    M["Researcher Agent"]
    N["Analyst Agent"]
    K <--> L
    K <--> M
    K <--> N
end

N --> O["Decision + Action\n(+ Trace ID)"]
O --> F
F -->|"Outcomes + rewards"| P

subgraph Learning ["Self-Improvement Loop (every 5 min)"]
    P["Learning Loop\n(learning_loop.py)"]
end

P -->|"Bandit Q-update"| G
P -->|"Confidence decay"| I

Q["Human Feedback\n(run_agent.py feedback)"] -->|"score + correction"|
R
R["FeedbackManager\n(feedback.py)"] -->|"New facts"| C
R -->|"Reward signal"| G

```

4.2 Memory Subsystem Detail

```

flowchart LR
    subgraph Graph ["Entity Graph (Neo4j)"]
        E1["Entity Node\nname: Tesla\ntype: Company\nconfidence:
0.82\nupdated: 2026-02-22"]
        E2["Entity Node\nname: Earnings"]
        E3["Entity Node\nname: Rally"]
        E1 -->|"catalyst\nts: 2026-02-22\nconf: 0.9"| E2
        E1 -->|"expected_movement\nts: 2026-02-22\nconf: 0.42"| E3
    end

    subgraph Vector ["Vector Store (ChromaDB)"]
        V1["Chunk ID: abc123\n'Tesla stock will rally...\nembedding:
[...]" ]
    end

```

```

subgraph Cluster ["Cluster Index (HDBSCAN)"]
  C0["Cluster 0\n[Tesla, AMZN, AAPL]\n(Tech-Growth)"]
  C1["Cluster 1\n[Earnings, Revenue, Q4]\n(Financial Events)"]
  CN["Noise\n(outlier entities)"]
end

E1 -->|"embedding"| C0
E2 -->|"embedding"| C1
V1 -->|"source_id link"| E1

```

4.3 AgentMem Controller State Machine

```

stateDiagram-v2
    [*] --> Idle

    Idle --> Evaluating : Query arrives

    Evaluating --> Retrieve : Always
    Retrieve --> BanditSelect : UCB arm selection
    BanditSelect --> NarrowRetrieve : Arm: narrow (depth=1, breadth=3)
    BanditSelect --> MediumRetrieve : Arm: medium (depth=1, breadth=5)
    BanditSelect --> WideRetrieve : Arm: wide (depth=2, breadth=10)
    BanditSelect --> DeepRetrieve : Arm: deep (depth=3, breadth=15)

    NarrowRetrieve --> Decide
    MediumRetrieve --> Decide
    WideRetrieve --> Decide
    DeepRetrieve --> Decide

    Decide --> StoreCheck : New facts extracted
    StoreCheck --> Store : confidence ≥ 0.4
    StoreCheck --> Defer : confidence < 0.4

    Store --> LogTrace
    Defer --> LogTrace
    LogTrace --> Idle

    Idle --> Decay : TTL timer fires (every 5 min)
    Decay --> Idle

```

4.4 Self-Improvement Learning Loop

```

sequenceDiagram
    participant U as User
    participant W as Workflow
    participant C as AgentMemController
    participant E as EpisodicStore
    participant L as LearningLoop

```

```

U->>W: "What is Tesla's outlook?"
W->>C: get_retrieval_params()
C-->>W: {depth: 1, breadth: 5} [UCB selects 'medium']
W->>E: log_trace(trace_id, outcome="unknown")
W-->>U: Decision + Trace ID: abc-123

```

Note over U,L: ...time passes...

```

U->>W: feedback --trace-id abc-123 --score 0.9
W->>E: update_outcome(abc-123, "success", reward=0.9)

```

```

loop Every 5 minutes
  L->>E: get_traces_for_learning()
  E-->>L: [ContextTrace(reward=0.9, arm="medium")]
  L->>C: register_reward(0.9)
  C-->>L: Q('medium') updated → 0.9
  L->>GraphStore: apply_confidence_decay(factor=0.9)
end

```

Note over C: Next query – UCB prefers 'medium' arm

4.5 Multi-Agent GroupChat

```

flowchart TD
    U["User Query"] --> UP["UserProxy"]

    UP --> GC

    subgraph GC ["GroupChat (GroupChatManager – GPT-4o)"]
        direction TB
        PL["📁 Planner\nBreaks query into subtasks\nCan hire new agents"]
        MS["🧠 Memory Specialist\nretrieve_memory · graph_editor"]
        RE["🔍 Researcher\nweb_search"]
        AN["🇮🇹 Analyst\ncalculator · final synthesis"]

        PL -- "delegates" --> MS
        PL -- "delegates" --> RE
        PL -- "delegates" --> AN
        PL -- "hire_agent()" --> DYN
        DYN["⚡ Dynamic Agent\n(spawned at runtime)"]
    end

    AN --> UP
    UP --> U

```

5. Component Reference

Component	File	Responsibility
Ingestor	src/processing/ingest.py	Canonicalize text → entities + triplets
GraphStore	src/memory/graph_store.py	Neo4j driver; typed, timestamped edges
VectorStore	src/memory/vector_store.py	ChromaDB client; semantic embedding search
ClusterStore	src/memory/cluster_store.py	HDBSCAN cluster index; soft expansion
EpisodicStore	src/memory/episodic_store.py	Append-only trace log with outcome support
Retriever	src/processing/retrieval.py	3-phase recall: vector + graph + cluster
AgentMemController	src/agent/agentmem_controller.py	UCB bandit policy over memory ops
LearningLoop	src/agent/learning_loop.py	Background TTL decay + reward updates
FeedbackManager	src/agent/feedback.py	Human corrections → graph write-back
AutoGenWorkflow	src/agent/workflow_autogen.py	GroupChat orchestration

6. Data Schemas

ContextTrace (Full Provenance Record)

```
class ContextTrace(BaseModel):
    trace_id: str # UUID
    task_id: str
    input_query: str
    retrieved_memory_ids: List[str] # IDs of all memory objects
    retrieved
    graph_paths: List[List[str]]
    reasoning_steps: List[str]
    final_decision: str
    confidence: float
    outcome: Literal["success", "failure", "unknown"]
    reward_signal: float # [-1.0, 1.0] ← drives bandit
    timestamp: datetime
    feedback: Optional[Feedback]
```

Triplet (Knowledge Graph Edge)

```
class Triplet(BaseModel):
    subject: str # "Tesla"
    predicate: str # "expected_movement"
    object: str # "rally"
    timestamp: str
    confidence: float # decays over time via LearningLoop
    source_id: str # links back to VectorStore chunk
```

7. Operational Guide

Quick Start (Docker — Recommended)

```
git clone <repo>
cd agent_memory_system
cp .env.example .env # set OPENAI_API_KEY
docker-compose up --build
```

All three services start with healthcheck ordering:

- **Neo4j** → available at **localhost:7474** (browser) and **localhost:7687** (Bolt)
- **ChromaDB** → available at **localhost:8000**
- **Agent** → interactive CLI once both databases are healthy

CLI Reference

```
# Interactive real-time mode (with news monitor + learning loop)
python start_agent.py

# One-off ingestion
python run_agent.py ingest --text "Apple Q4 revenue was $120B"

# One-off query
python run_agent.py query "What is Apple's financial outlook?"

# Submit feedback on a past decision
python run_agent.py feedback --trace-id <uuid> --score 0.9 \
    --correction "Apple is actually in a strong position due to services growth"

# Inspect bandit arm Q-values (in start_agent.py interactive mode)
> bandit
```

Environment Variables

Variable	Default	Description
----------	---------	-------------

Variable	Default	Description
OPENAI_API_KEY	—	Required for LLM calls
NEO4J_URI	bolt://localhost:7687	Graph DB connection
NEO4J_USER	neo4j	Graph DB username
NEO4J_PASSWORD	password	Graph DB password
CHROMA_HOST	localhost	Vector DB host
CHROMA_PORT	8000	Vector DB port

8. Self-Improvement Mechanics

Phase 1 — Heuristics (Active)

The simplest improvement mechanism: old facts decay, high-confidence facts persist.

$$\text{confidence}(t) = \text{confidence}(t_0) \times 0.9^n$$

where n = number of decay cycles since last reinforcement. Facts below 0.05 confidence are candidates for deletion.

Phase 2 — Bandit Learning (Active)

The **AgentMemController** maintains four retrieval arms:

Arm	Graph Depth	Entity Breadth	Use Case
narrow	1	3	Simple factual queries
medium	1	5	Standard queries
wide	2	10	Relational queries
deep	3	15	Complex multi-hop queries

UCB selection formula:

$$\text{arm*} = \operatorname{argmax}_a \left[Q(a) + \sqrt{2 \ln(N) / n(a)} \right]$$

where $Q(a)$ is the average reward for arm a , N is total pulls, $n(a)$ is pulls for arm a .

Phase 3 — GRPO / RLVR (Planned)

Offline optimization over episodic trace batches. Reward shaping from outcomes + human feedback. No end-to-end gradient through the memory store.

9. Complexity Analysis

Operation	Complexity	Notes
Entity write	$O(\log N)$	Neo4j B-tree index on name
Vector search	$O(\log N)$	ChromaDB HNSW index
Graph traversal	$O(k \cdot d)$	k neighbors per hop, d hops
Cluster expansion	$O(K)$	K = cluster size, bounded
TTL decay	$O(N)$	Full graph scan, runs offline
Bandit arm select	$O(A)$	A = 4 arms, effectively $O(1)$

10. Failure Modes & Mitigations

Risk	Likelihood	Mitigation
Memory bloat from ingestion spam	Medium	AgentMem confidence gate ($DEFER < 0.4$) + TTL decay
Retrieval noise corrupts reasoning	Medium	Cluster expansion budget cap (top 5 entities)
Bandit feedback loop collapse	Low	Epsilon-greedy exploration ($\epsilon = 0.15$); offline-only updates
Neo4j unavailable at boot	Medium	<code>depends_on: healthcheck</code> in <code>docker-compose.yml</code>
Graph edge proliferation	Low	Relationship deduplication via MERGE in Neo4j Cypher
Human feedback poisons memory	Low	FeedbackManager re-canonicalizes correction through same pipeline as ingest

11. Testing

```
# Run all v2 tests
python -m unittest tests/test_agentmem_v2.py -v

# Run full suite
python -m unittest discover tests/ -v
```

Test Class	Coverage
<code>TestClusterStore</code>	add, update, len, expand_query
<code>TestAgentMemController</code>	store gate, defer gate, UCB selection, reward update
<code>TestLearningLoop</code>	empty batch, reward propagation

Test Class	Coverage
TestDynamicAgents	hire_agent creates + adds to GroupChat

12. Roadmap

- **v2.1** — Persistent ClusterStore (SQLite sidecar for cluster memberships across restarts)
- **v2.2** — Embed query strings for true semantic cluster expansion (replace zero-vector proxy)
- **v2.3** — Phase 3 offline GRPO update over episodic trace batches
- **v3.0** — Multimodal ingestion (image → entity attributes, chart → structured signals)